Lec-11

22/03/26 9:52 AM

5 sec
3 sec

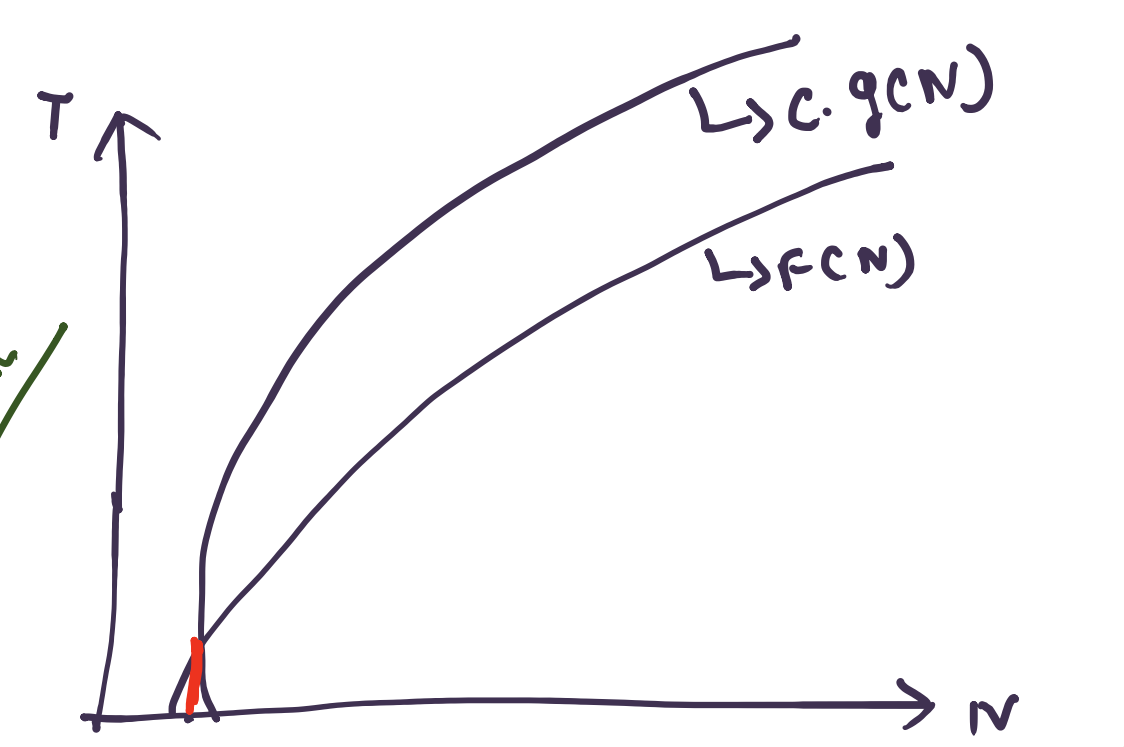
Time & ~~Space~~ Complexity

30-40 ×
① ✓
(1 sec) sec
2 → loop
TLE

asymptotic notation vs Experimental Way
①

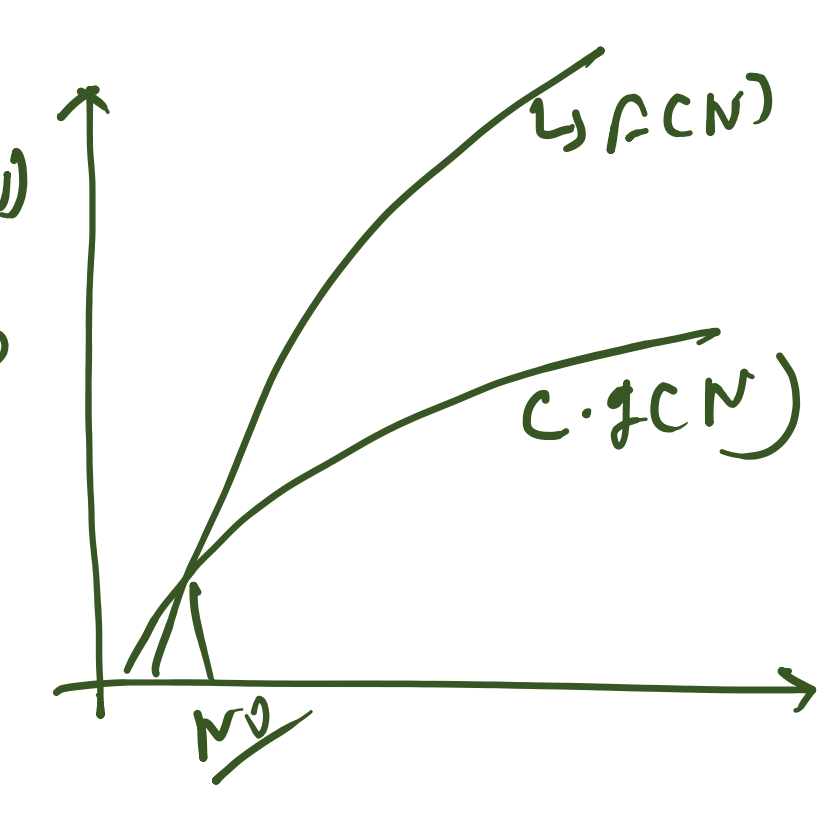
Worst Case
× Best case
× Avg case

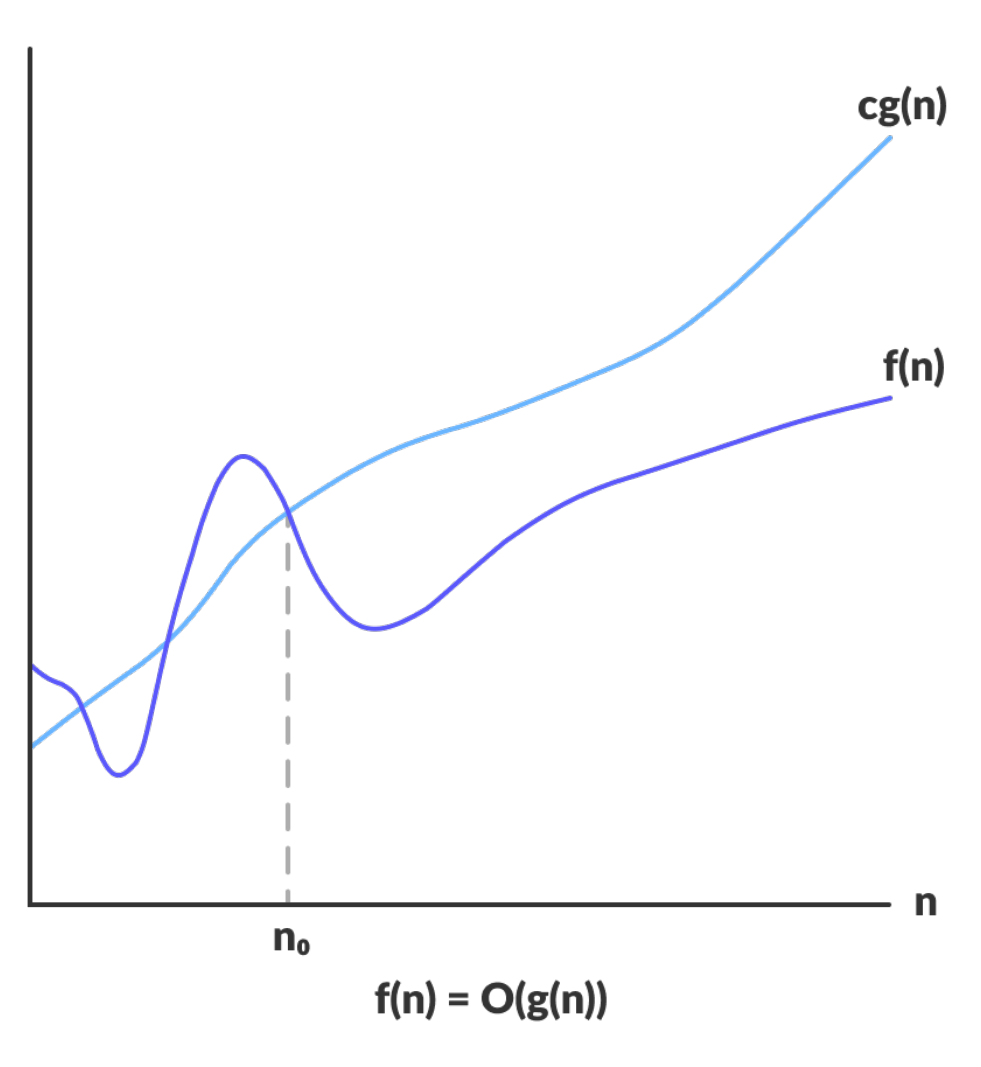
O(g(n))
Big-oh



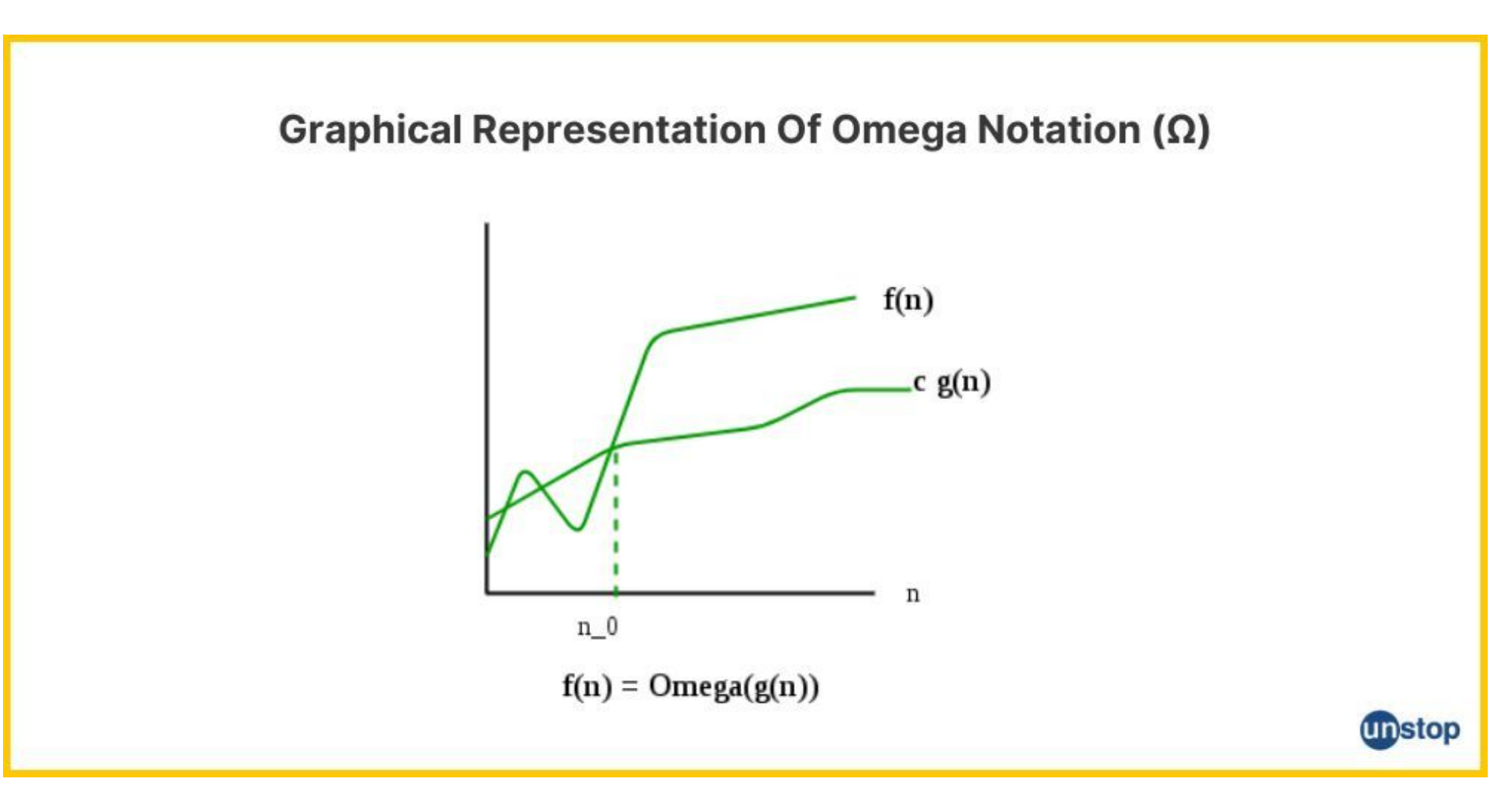
F(N) ≤ c.g(N) N≥No

F(N) >= c.g(N)
N ≥ No
Ω(g(N))





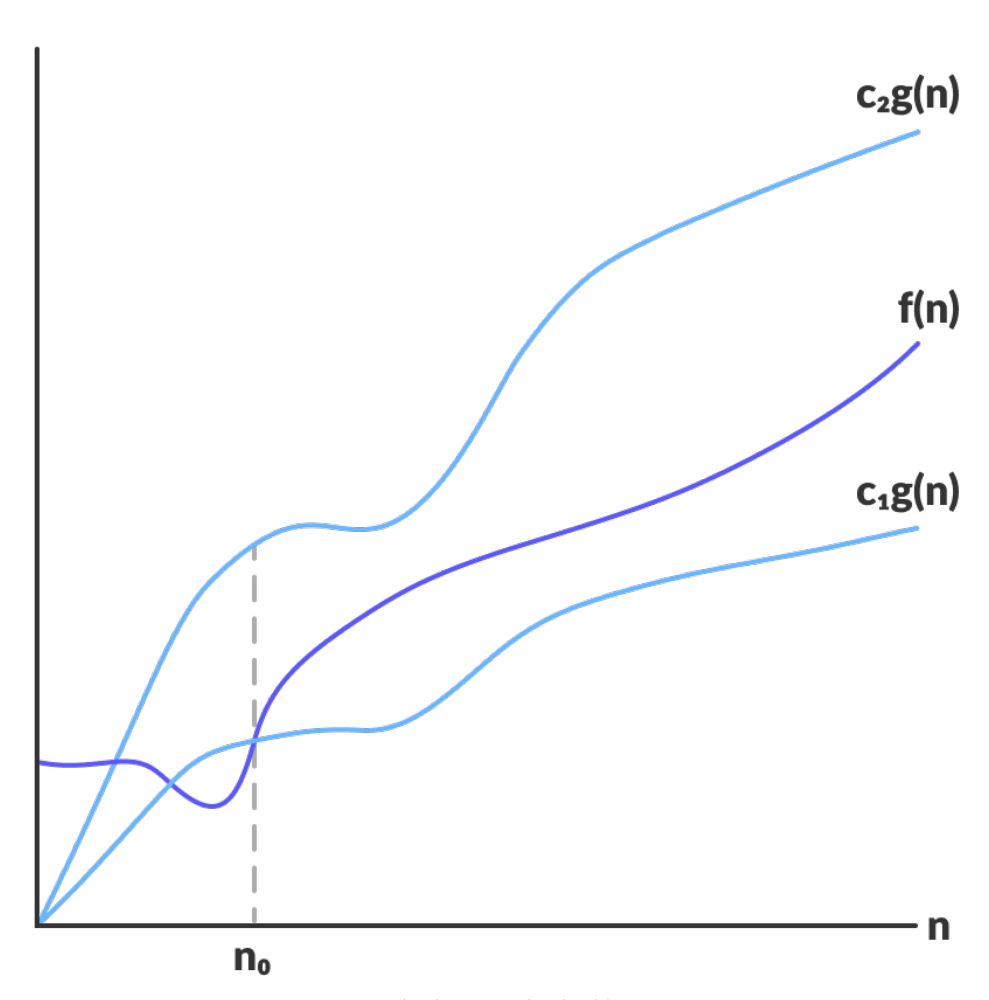
f(n) = O(g(n))



Graphical Representation Of Omega Notation (Ω)

f(n) = Omega(g(n))

O
Ω
Θ
Big Oh
Big Omega
Big Theta



f(n) = Θ(g(n))

10 → 20
50 → 4f

F(N) ≤ c.g(N)
F(N) = 5N² + 7N + 4
c.g(N) = 16 N²

T(N) = 5N² + 7N + 4

n²

O(n²)

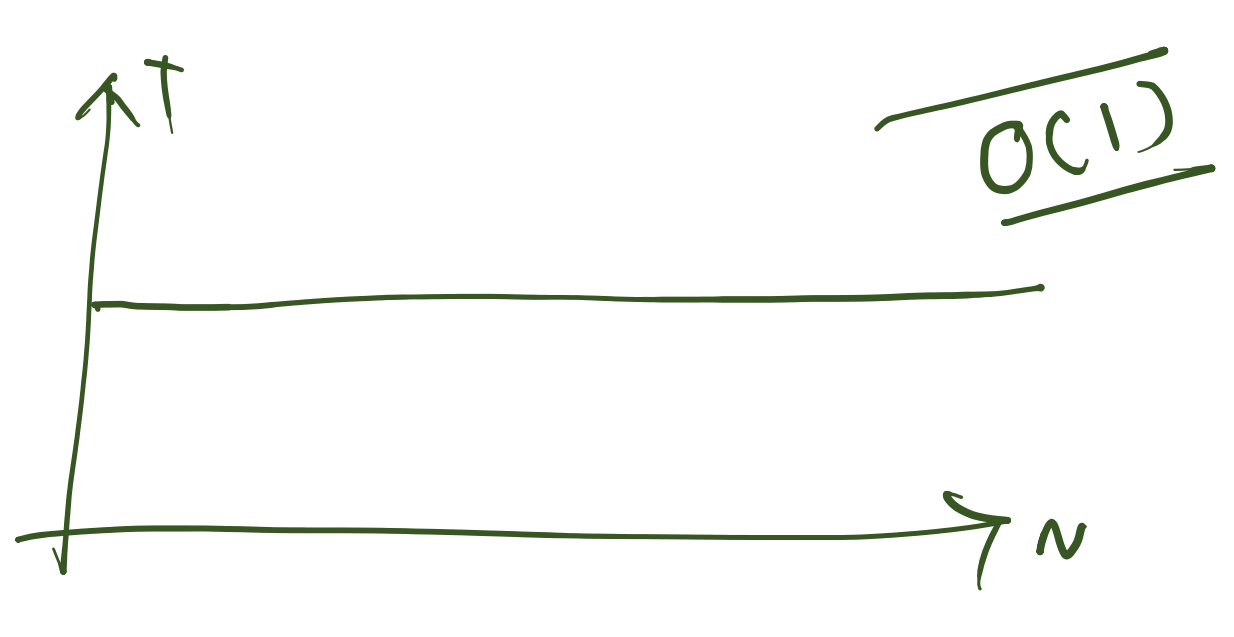
F(N) ≤ c.g(N²) g(n) = n²
5N² + 7N + 4 ≤ ⑯ N²

T(N) = 7N³ + 2ᴺ + 2N logN + 5n⁴ + 7 + log(N)

Class ✓

2ᴺ × log(N)

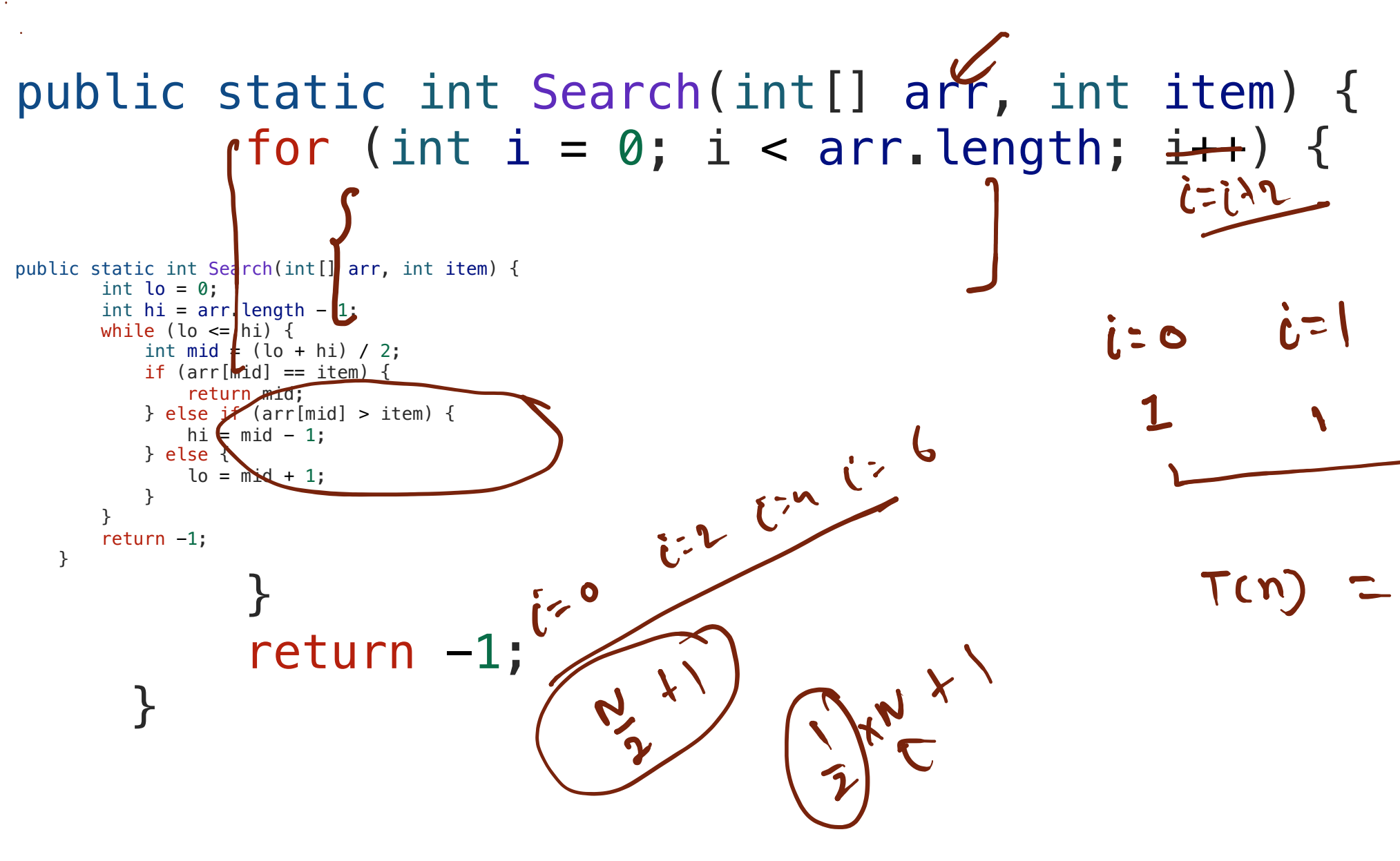
Constant time



O(1)

T.C

① ✓ Single Point ⇒ O(1)



```java
public static int Search(int[] arr, int item) {
    for (int i = 0; i < arr.length; i++) {
```

i = i + 2

```java
public static int Search(int[] arr, int item) {
        int lo = 0;
        int hi = arr.length - 1;
        while (lo <= hi) {
            int mid = (lo + hi) / 2;
            if (arr[mid] == item) {
                return mid;
            } else if (arr[mid] > item) {
                hi = mid - 1;
            } else {
                lo = mid + 1;
            }
        }
        return -1;
    }
```

```java
    }
    return -1;
}
```

<= 10⁵

i=0   i=1   i=2   i=3   - - -   i=n-1
1   1   1   1   1

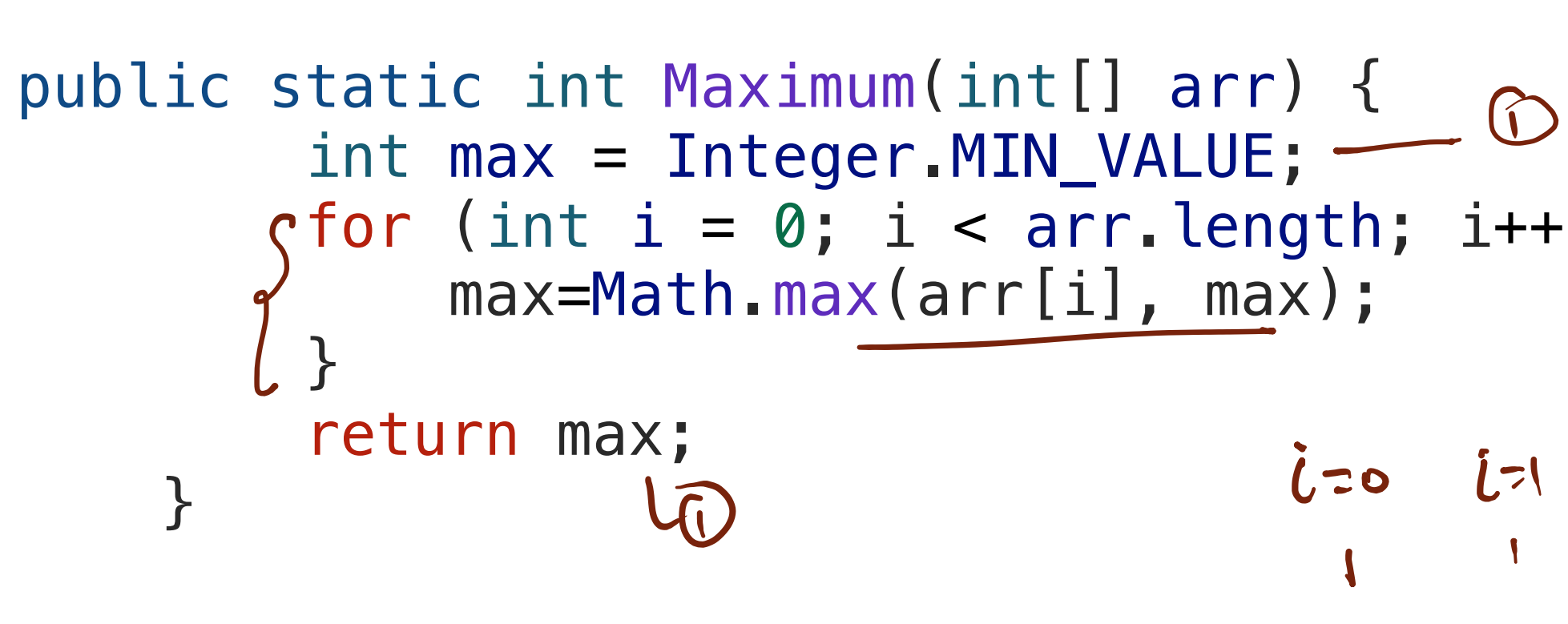
T(n) = N + 1

O(n)

i=0 i=2 i=4 i=6

N/2 + 1

½ × N + 1
c

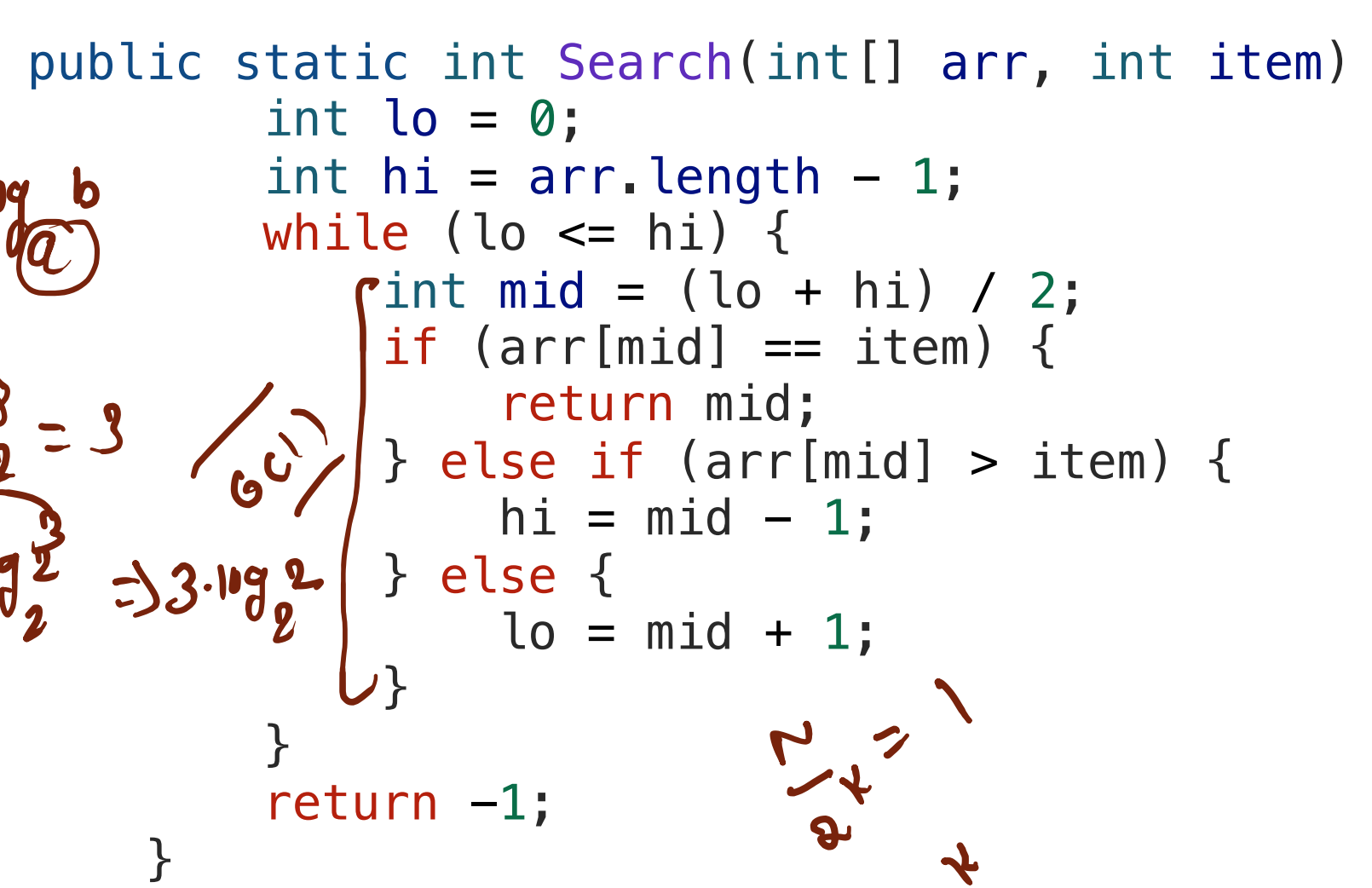


```java
public static int Maximum(int[] arr) {
        int max = Integer.MIN_VALUE;
        for (int i = 0; i < arr.length; i++) {
            max=Math.max(arr[i], max);
        }
        return max;
    }
```

①
①

i=0 i=1 i=2 - - - i=n-1
1 1 1 1

T(n) = (N+2)



```java
public static int Search(int[] arr, int item) {
        int lo = 0;
        int hi = arr.length - 1;
        while (lo <= hi) {
            int mid = (lo + hi) / 2;
            if (arr[mid] == item) {
                return mid;
            } else if (arr[mid] > item) {
                hi = mid - 1;
            } else {
                lo = mid + 1;
            }
        }
        return -1;
    }
```

$\log_a b$

$\log_2 8 = 3$

$\log_2 2^3 \Rightarrow 3 \cdot \log_2 2$

O(1)

N → 1
N/2 → 1
N/2² → 1
N/2³ → 1
N/2⁴ → 1
N/2⁵ → 1
(N/2ᵏ = 1) → 1

T.C = O(K) ?

T.C = $\log_2 N$

N/2ᵏ = 1

N = 2ᵏ

2ᵏ = N

K = $\log_2 N$

2ᵏ = N
K = $\log_2 N$

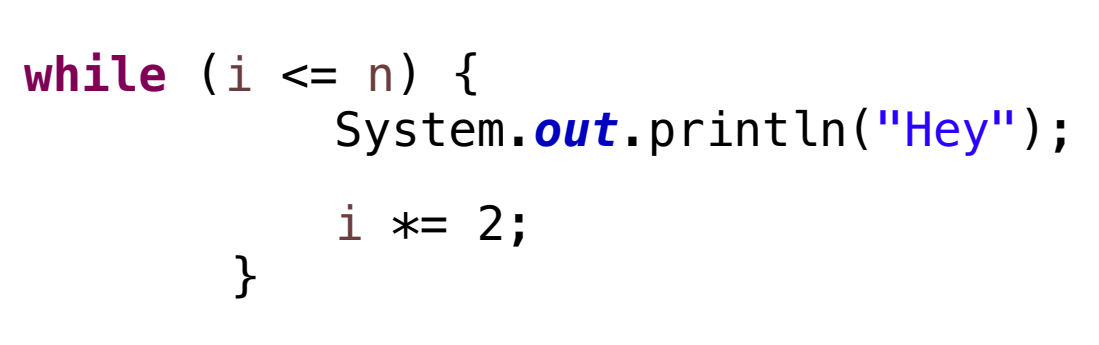
while (n > 0) {

n = n/2

i=1 i=2 i=2² i=2³ i=2⁴

2ᵏ = N

```java
while (i <= n) {
            System.out.println("Hey");
            i *= 2;
        }
```

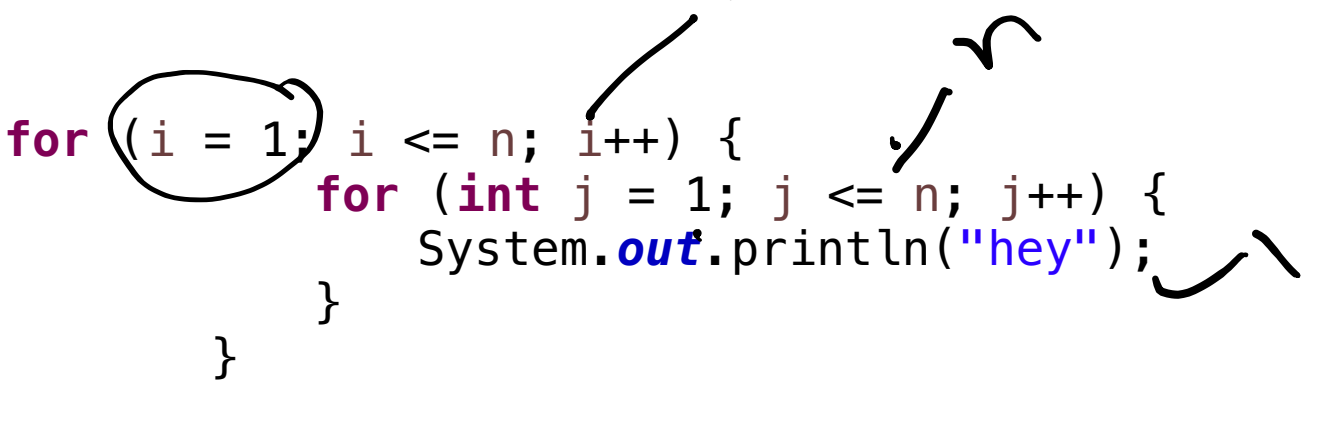


2ᵏ = N
K = $\log_2 N$

i=1 i=2 i=2² i=2³ i=2⁴ - - - i=2ᵏ=n
1 1 1 1 1 1

K

n × n × 1 = n²

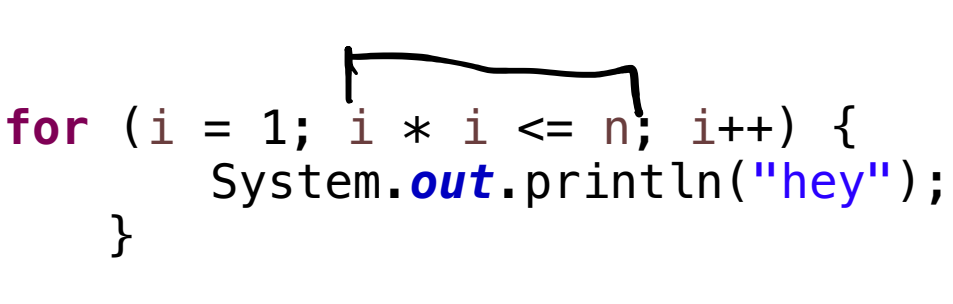


```java
for (i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                System.out.println("hey");
            }
        }
```

n

i=1 i=2 i=3 - - - i=N
N N N N

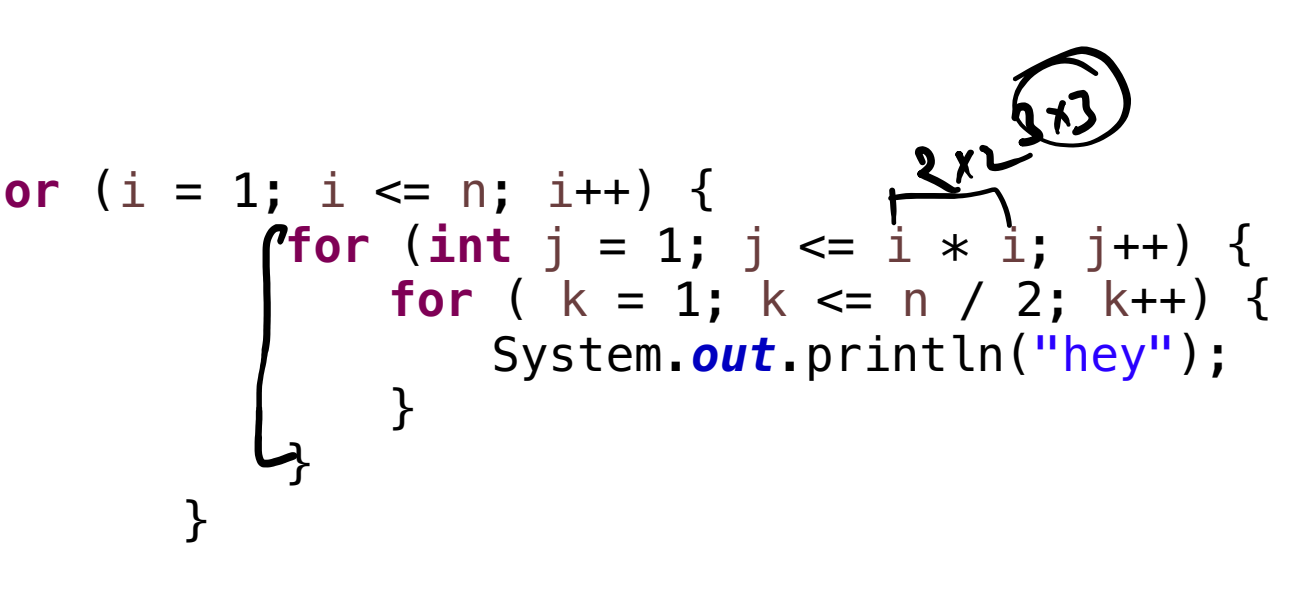
n²



```java
for (i = 1; i * i <= n; i++) {
            System.out.println("hey");
        }
```

i² ≤ N
i ≤ √N

Sq(N)



```java
for (i = 1; i <= n; i++) {
            for (int j = 1; j <= i * i; j++) {
                for ( k = 1; k <= n / 2; k++) {
                    System.out.println("hey");
                }
            }
        }
```

2×2 3×3

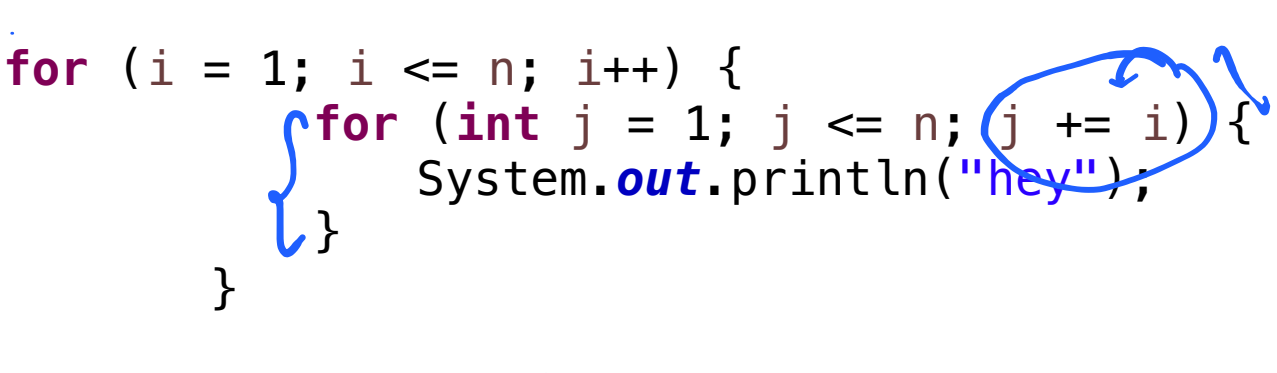
N/2 N/2 N/2 N/2

i=1 i=2 i=3 i=4 - - - i=n
N/2 2² × N/2 3² N/2 4² N/2 n² N/2

$\frac{N}{2}[1^2 + 2^2 + 3^2 + 4^2 + \dots + n^2]$

$\frac{N}{2}\left[\frac{N(n+1)(2n+1)}{6}\right]$

T.C n⁴



```java
for (i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j += i) {
                System.out.println("hey");
            }
        }
```

i=1 i=2 i=3 i=4 - - i=n
N N/2 N/3 N/4 N/N

N (1 + 1/2 + 1/3 + 1/4 - - - 1/N)
→ log(N)

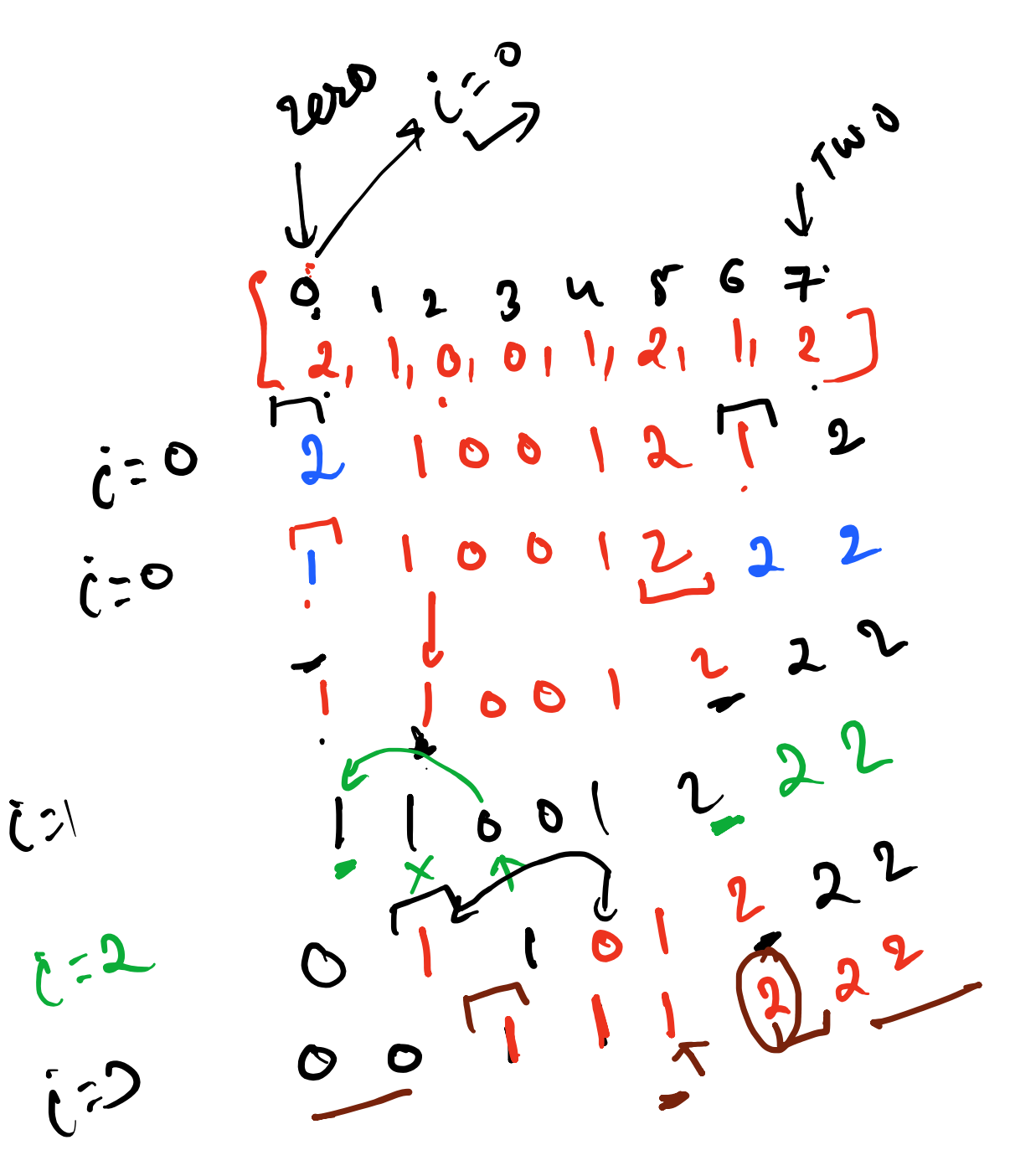
12th Class
M1

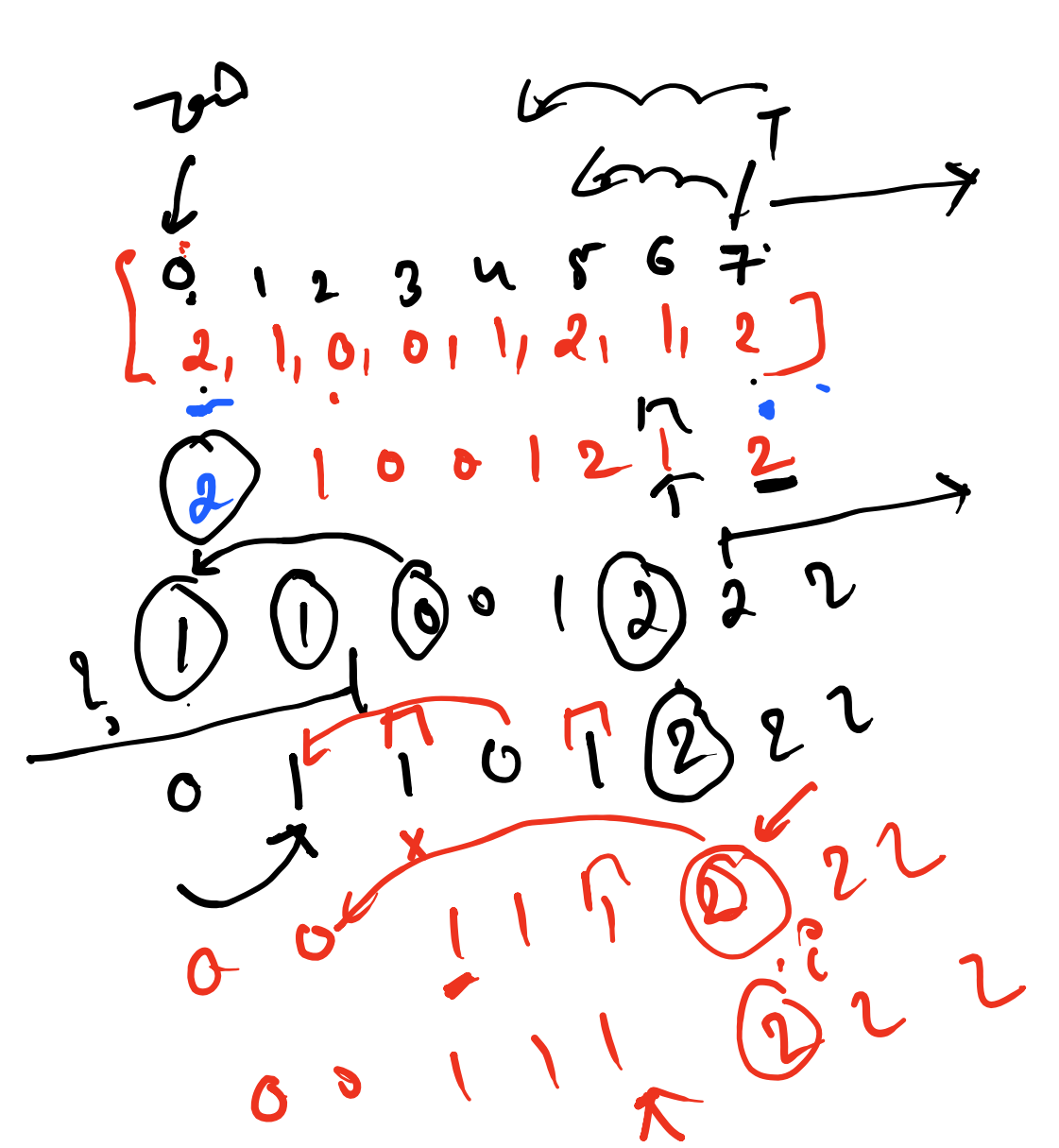
N × log(N)

0 0111 222

0 → 2
1 → 3
2 → 2

Red → 1
pink → 2
white → 0





while (i <= Two) {
if (arr[i] == 0) {
swap (zero, i)
zero++ i++
}
else if (arr[i] == 2) {
swap (Two, i)
Two--
}
else {
i++
}
}